MATHEMATICS ROADMAP

Computer Science + Machine Learning

A single, ordered study plan for rebuilding mathematical fluency after a long break. The core path emphasizes understanding and problem solving; programming is optional, not a gate you must pass.

The recommended sequence

Work through the stages in order, but treat the early stages as a diagnostic refresher. Since you have studied university-level engineering mathematics before, move quickly through anything you can still solve confidently.

01	Algebra, functions, geometry and trigonometry Rebuild the language and prerequisites.
02	Single-variable calculus Limits, derivatives, integrals and series essentials.
03	Discrete mathematics for CS Logic, proofs, induction, graphs, counting, recurrences and Big-O.
04	Linear algebra Vectors, matrices, vector spaces, projections, eigenvalues and SVD.
05	Multivariable calculus Partial derivatives, gradients, chain rule and optimization geometry.
06	Probability and statistics Random variables, distributions, inference, hypothesis testing and regression.
07	Optimization for ML Objective functions, convexity, gradient methods and regularization.
08	ML mathematics synthesis (optional applied bridge) Connect the math topics to supervised learning and neural networks.

Study pace

At roughly 4-6 hours per week, allow around 8-14 months for a careful pass through the core sequence. This is a planning range, not a deadline: a refresher can be much faster if the material comes back. Do problems rather than trying to finish every lecture.

Stage 1 - Rebuild the foundations

01

Algebra, functions, geometry and trigonometry

Suggested pace: 2-5 weeks, selectively

Primary resource: OpenStax Precalculus 2e (free textbook) | Vocabulary/practice support: Khan Academy Math

Study these topics

- Fractions, powers, roots, factoring, equations and inequalities; rearranging formulas.
- Functions, domain and range, graphs, composition, inverses, polynomial and rational functions.
- Exponentials and logarithms, including log rules.
- Angles in radians, unit circle, sine/cosine/tangent, basic identities and graphing periodic functions.
- Coordinate geometry, vectors, dot product, distance and lines/planes at an introductory level.

How to use the resources: Do not read the whole textbook by default. Use the chapter review tests or Khan Academy topic quizzes to locate weak areas, then study only those sections and solve exercises until the steps feel natural.

Checkpoint before moving on
You can solve common algebraic equations without searching for a formula, sketch basic function families, manipulate exponents/logarithms, interpret a graph, and work comfortably with radians and the unit circle.

Stage 2 - Single-variable calculus

02

Calculus: differentiation and integration

Suggested pace: 4-8 weeks as a refresher; longer if needed

Primary resource: MIT OpenCourseWare 18.01SC - Single Variable Calculus

Study these topics

- Limits and continuity; derivative as rate of change and local linear approximation.
- Product, quotient and chain rules; implicit differentiation; derivatives of common functions.
- Definite and indefinite integrals, the Fundamental Theorem of Calculus, substitution and integration by parts.
- Taylor approximation and infinite series at a practical conceptual level; revisit more deeply only if needed.

For CS and ML: Focus on what a derivative means, how it changes under composition, and how integrals accumulate quantities. You do not need to redo every engineering application or memorize every integration trick.

Checkpoint before moving on
Given a function, you can differentiate it, interpret the derivative, locate a simple maximum/minimum, evaluate basic integrals, and explain why the chain rule is central to nested computations.

Stage 3 - Discrete mathematics for computer science

03

Mathematical reasoning for algorithms and software

Suggested pace: 6-12 weeks

Primary resource: MIT OpenCourseWare 6.042J - Mathematics for Computer Science

Study these topics

- Propositional logic, predicates, quantifiers, sets, functions, relations and equivalence relations.
- Proof techniques: direct proof, contradiction, contrapositive, induction and strong induction.
- Counting: permutations, combinations, pigeonhole principle and inclusion-exclusion.
- Number theory and modular arithmetic; divisibility, remainders and why these matter for hashing and cryptography.
- Graphs, trees, paths, connectivity, invariants and state-machine reasoning.
- Sequences, recurrences, summations and asymptotic growth (Big-O, Big-Theta and Big-Omega).
- Basic discrete probability and expected value.

Why this is a core CS phase: This material supports algorithm analysis, data structures, correctness arguments, graph algorithms, complexity and later theoretical CS. It is not a substitute for calculus; it develops a different kind of mathematical reasoning.

Checkpoint before moving on
You can prove a simple claim by induction, reason with quantifiers, count outcomes, explain a graph or tree property, solve a basic recurrence, and distinguish an algorithm that is O(n) from one that is O(n^2).

Stage 4 - Linear algebra

04

Vectors, matrices and linear transformations

Suggested pace: 5-9 weeks

Primary resource: MIT OpenCourseWare 18.06SC - Linear Algebra

Study these topics

- Vectors, matrices, matrix multiplication and systems of linear equations; Gaussian elimination, rank and the four fundamental subspaces.
- Linear independence, basis, dimension, span and linear transformations.
- Orthogonality, projections, least squares and geometric interpretation of regression.
- Determinants, eigenvalues/eigenvectors, diagonalization and singular value decomposition (SVD).

For ML: Spend particular time on dot products, matrix-vector multiplication, projections, least squares, eigenvectors and SVD. Understand the geometry and meaning, not only how to multiply matrices.

Checkpoint before moving on
You can solve Ax = b, explain rank and linear independence, describe a matrix as a transformation, derive least-squares fitting, and explain what eigenvectors and singular values reveal.

Stage 5 - Multivariable calculus

05

Gradients and functions of several variables

Suggested pace: 4-8 weeks, focusing on ML-relevant topics

Primary resource: MIT OpenCourseWare 18.02SC - Multivariable Calculus

Study these topics

- Functions of several variables, surfaces and contours; partial derivatives.
- Gradient, directional derivatives, chain rule and local linear approximation.
- Critical points, Hessian intuition, minima/maxima and constrained optimization basics.
- Jacobian matrices and multivariable chain rule; become comfortable with the notation for vector-valued functions.
- Multiple integrals and vector calculus are valuable for breadth, but can be selective for a standard ML path.

For ML: The key idea is how changing many inputs changes one objective. The gradient indicates the direction of steepest increase; optimization methods use it to move toward lower loss.

Checkpoint before moving on
You can compute and interpret a gradient, apply the chain rule to a composed multivariable function, and describe how gradients guide the search for a minimum.

Stage 6 - Probability and statistics

06

Uncertainty, inference and data

Suggested pace: 6-12 weeks; study probability before inference

Probability: Harvard Statistics 110 - Probability
Statistics: OpenStax Introductory Statistics 2e (free textbook)

Probability first

- Sample spaces, conditional probability, independence, Bayes' theorem and counting-based probability.
- Random variables, expectation, variance, covariance and correlation.
- Bernoulli, binomial, normal, Poisson and other common distributions; joint and conditional distributions.
- Law of large numbers, central limit theorem and sampling distributions.

Then statistical inference

- Population vs sample, descriptive statistics, bias, sampling and experimental design.
- Point estimates, confidence intervals, hypothesis tests, p-values and comparing groups.
- Correlation and linear regression; interpret the assumptions and limitations of results.

For ML: Prioritize conditional probability, distributions, expectation/variance, sampling, estimation, regression and the difference between prediction and inference. No statistical software is required to learn these foundations.

Stage 7 - Optimization for machine learning

07

From objective functions to learning algorithms

Suggested pace: 3-6 weeks for foundations; deeper study is optional

Primary reference: Boyd and Vandenberghe - Convex Optimization (free online book)

Study these topics

- Objective/loss functions, decision variables, constraints and feasible sets.
- Convex sets and convex functions; why convexity can make optimization easier.
- Gradient descent, learning rate, convergence intuition, Newton's method and second-order information.
- Lagrange multipliers and constrained optimization at an introductory level.
- Regularization and the relationship among least squares, loss functions and model fitting.

Use the book selectively. You do not need to master the entire convex optimization text before starting ML. Concentrate first on the geometry of optimization, gradients, and why an algorithm makes progress or fails.

Checkpoint before moving on
You can formulate a simple minimization problem, explain gradient descent in words and equations, identify how the learning rate affects an update, and explain what regularization is trying to control.

Stage 8 - Connect the mathematics to ML

08

An optional applied synthesis

Suggested pace: 4-8 weeks; optional, not a prerequisite

Optional course: DeepLearning.AI - Mathematics for Machine Learning and Data Science

Use this as a bridge from the core mathematics to common ML ideas. Revisit linear regression, vectors and transformations, derivatives and gradients, probability distributions, estimation and hypothesis testing. This specialization includes Python labs; they are optional for this roadmap, and you can skip the programming exercises if your goal at this stage is mathematical understanding.

Final synthesis tasks (paper-and-pencil is enough)

- Derive the least-squares objective for linear regression and explain its geometric interpretation.
- Write one gradient-descent update for a simple loss function and explain each term.
- Explain Bayes' theorem and give a real example of how a prior and evidence combine.
- Explain PCA conceptually using projections, variance and eigenvectors/SVD.
- Read a confusion matrix and interpret precision, recall and the trade-off between them.

Make the roadmap practical

Use the same simple study loop at every stage. It works whether you follow a full university course or use the material as a refresher.

- Use one main resource per stage. Add Khan Academy only to repair a specific prerequisite; do not juggle multiple full courses.
- Solve problems before watching solutions. Keep a notebook of definitions, worked examples and errors you repeatedly make.
- Use a mastery gate, not a calendar. Move on when you can solve representative problems and explain why the method works.
- Keep coding optional. Pencil-and-paper work is sufficient for the core sequence. Python can later visualize matrices, probability and gradients; no R-based course or workflow is part of this roadmap.
- Do not restart everything from zero. In the foundation, calculus and linear algebra stages, diagnose first and study weak topics selectively.

A simple weekly rhythm

Session A Learn one concept; write definitions and a worked example.	Session B Solve problems without looking at solutions.	Session C Correct mistakes and explain the idea from memory.

Core resource index

Stage	Main resource	Main purpose
1	OpenStax Precalculus 2e	Refresh prerequisites
2	MIT 18.01SC Calculus	Derivatives and integrals
3	MIT 6.042J Mathematics for CS	Proofs, graphs, Big-O
4	MIT 18.06SC Linear Algebra	Vectors and matrices
5	MIT 18.02SC Multivariable Calculus	Gradients and partial derivatives
6	Harvard Stat 110 + OpenStax Statistics	Probability and inference
7	Boyd and Vandenberghe	Optimization
8 optional	DeepLearning.AI math specialization	ML-focused synthesis; Python labs optional

What you should have at the end

A usable math foundation for algorithm analysis and CS theory, plus the linear algebra, calculus, probability, statistics and optimization needed to understand classical ML and the mathematical core of neural networks. For theoretical CS, add automata, computability and complexity. For deeper ML, add statistical learning theory and information theory. For scientific ML, revisit differential equations and numerical methods.